Asynchronous Programming and Null Safety in Dart

Assignment Report by Gaurav Kulkarni

Part 1: What I Understood and Learned

The main goal of this assignment was to build a Dart application that interacts with asynchronous data using Future, async, and await, while maintaining strict null safety and handling errors. Since I'm used to writing mostly synchronous code for my backend projects at ITM, shifting to an event-driven design took a minute to wrap my head around, but I definitely learned a lot.

Understanding Futures and the Event Loop

I learned that in Dart, a Future represents a value or error that will happen at some point in the future. Instead of freezing the whole app while waiting for an API response, Dart uses an internal Event Loop. Using the await keyword makes asynchronous code read just like normal step-by-step code, which is way easier to debug than nested callbacks. Using Future.delayed() to simulate network lag really helped me see how the app keeps running while waiting for data.

Working with Strict Null Safety

Dart's null safety is super strict. I learned how to set up my classes using nullable types (like String?) for things an API might not actually send back, like a bio or email. I also got really familiar with operators like the null-aware fallback (??), which I used to inject a default value of 0 if the followers count came back as null.

Designing Robust Data Serialization

In real life, APIs change or break, so I learned how to build a factory constructor (fromJson) that double-checks the incoming data before trying to create an object out of it. If an important key like 'id' is missing, it throws a FormatException instead of letting bad data break the rest of the app.

Part 2: Problems Faced and How I Fixed Them

Problem 1: App crashing on network errors

When I first tried throwing simulated network failures, my entire app just crashed with an 'Unhandled exception' and stopped running my other tests. **How I fixed it:** I wrapped all my API calls in a try-catch block. I specifically used 'on SocketException' for network drops and

'on FormatException' for bad data. I also added a finally block to make sure my logs finished printing whether it succeeded or failed.

Problem 2: Type-casting dynamic JSON maps

When parsing the JSON map, Dart treats the values as dynamic. When I tried to force a null value into an int, the app threw a massive type mismatch error. **How I fixed it:** I changed the logic to cast the incoming field as a nullable integer (as int?), and then used the ?? operator to fall back to 0. It completely stopped the runtime crashes.

Conclusion

Overall, this project really showed me why defensive programming is so important. By combining Dart's strict null safety checks with proper async/await error handling, I can now manage messy network calls without risking runtime crashes. This is going to be super useful when I start connecting real APIs to Flutter screens.